

1. Executive Summary

Pharmaceutical supply chains operate under strict regulatory, quality, and operational constraints that demand far higher levels of traceability, real-time monitoring, and impact analysis than traditional supply chain systems can provide. Temperature excursions, delayed quality control (QC) assessments, fragmented inventory visibility, and manual recall analysis directly impact patient safety, regulatory compliance, and revenue.

This project delivers an **end-to-end, event-driven pharmaceutical supply chain analytics platform** designed to address these challenges using a combination of **real-time streaming, batch analytics, and unified data modeling**. The platform continuously ingests temperature sensor data, sales transactions, inventory movements, and QC results, and correlates them through a common **batch-centric data spine**.

By combining **Spark Structured Streaming, batch ETL pipelines, and rule-driven alerting**, the platform enables:

- Immediate detection of cold chain violations
- Regulatory-grade batch traceability
- Proactive inventory replenishment decisions
- Near-instant recall impact assessment when QC failures occur

The result is a system that shifts pharmaceutical operations from **reactive reporting to real-time decision intelligence**, significantly reducing compliance risk, operational delays, and financial exposure.

1.1 Business Problem

Pharmaceutical supply chains face unique compliance and operational challenges that generic supply chain platforms are not designed to handle:

- **Cold chain integrity assurance**

Temperature-sensitive products such as vaccines, insulin, and biologics must be stored and transported within strict temperature ranges. Regulatory bodies require verifiable evidence that cold chain conditions were maintained continuously from manufacturing through distribution and dispensing.

- **Batch-level traceability as a regulatory requirement**

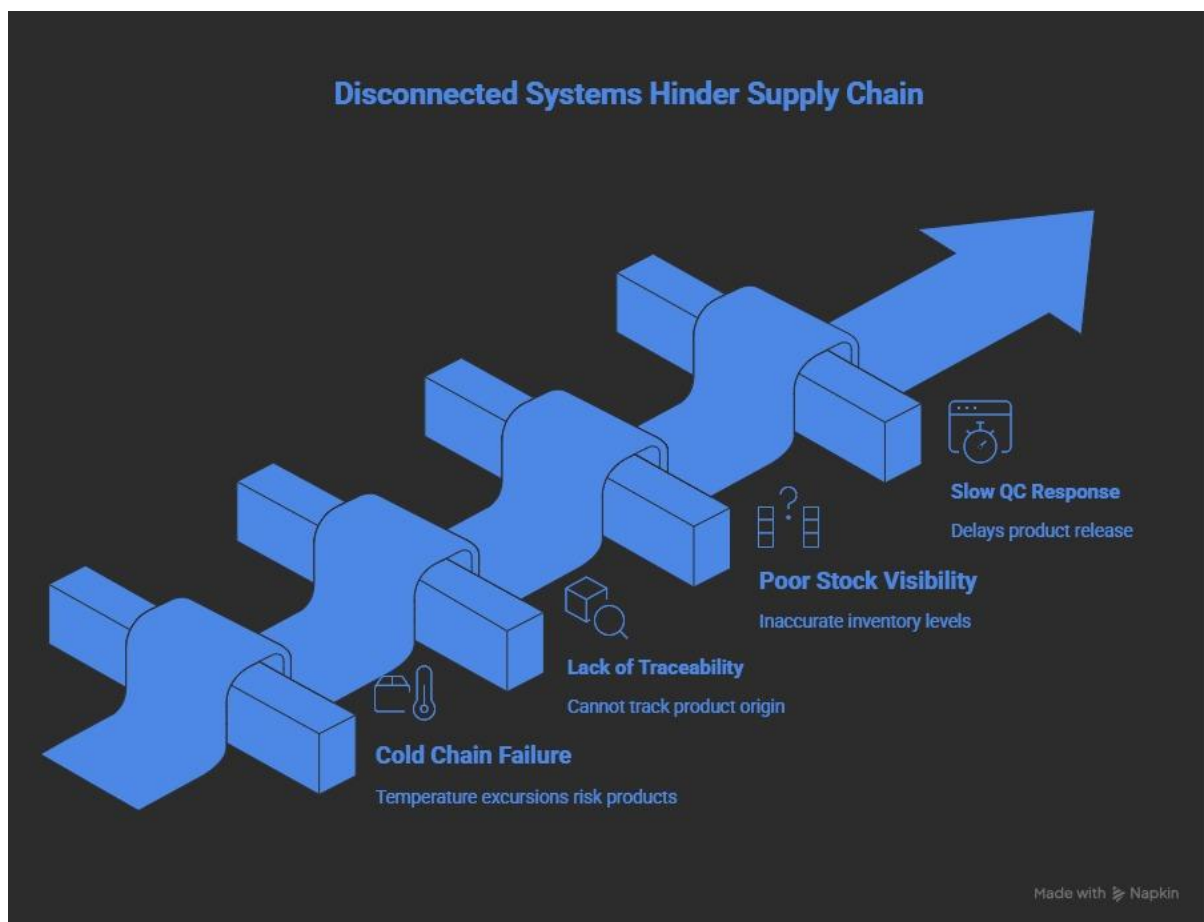
Every individual unit sold must be traceable back to its **manufactured batch**, along with associated QC test results, storage conditions, and movement history. Traditional systems often lack a unified mechanism to maintain this lineage end to end.

- **Fragmented distributor inventory visibility**

Inventory data is typically siloed across multiple distributors and warehouses. This fragmentation prevents early detection of stock-outs and near-expiry inventory, leading to unmet patient demand or avoidable write-offs.

- **Manual and delayed QC failure impact assessment**

When a batch fails post-release quality testing, organizations must identify affected orders, in-transit inventory, and revenue exposure. In many existing setups, this process is manual, error-prone, and can take days—during which unsafe products may continue moving through the supply chain.



1.2 Platform Intention

The platform is intentionally designed to address each of these business problems through tightly integrated streaming and batch analytics capabilities:

- **Real-time cold chain monitoring and alerting**

Temperature sensor readings are streamed continuously and evaluated in real time against the cold chain specifications defined at the batch level. Any breach is detected within seconds and escalated immediately to operations teams.

- **Unified batch-centric data spine**

A common batch_id is used as the central linking identifier across all datasets—temperature readings, sales transactions, QC results, and inventory movements—ensuring complete traceability throughout the product lifecycle.

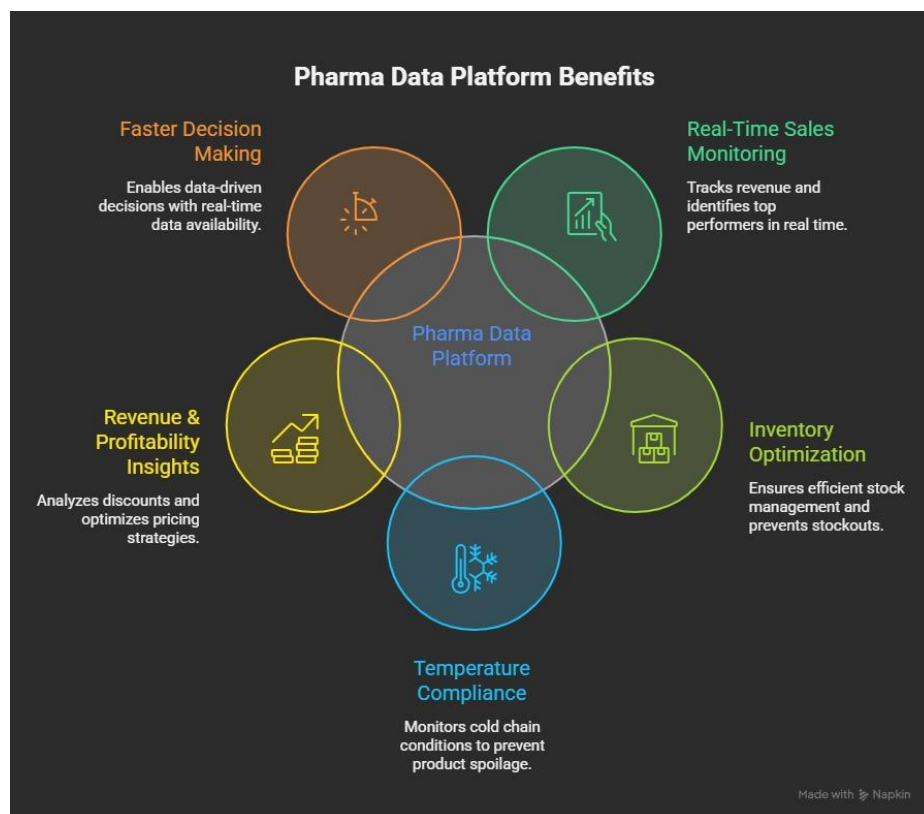
- **Continuous inventory coverage computation**

Real-time inventory movement events are joined with historical sales velocity to calculate live **days-of-stock coverage**, enabling early identification of replenishment risks before stock-outs occur.

- **Instant recall impact analysis**

QC failure records are automatically cross-referenced against live order fulfillment and inventory status to produce an immediate recall impact list, including affected orders and potential revenue at risk.

Together, these capabilities establish a **single, trusted, real-time intelligence layer** for pharmaceutical supply chain operations.



4.1 Streaming: Temperature Sensor Events

Source: IoT cold chain sensors. Transport: Kinesis. Frequency: Every 30 seconds per device.

Field	Type	Nullable	Description
event_id	String	No	Unique event identifier — primary deduplication key
event_time	Timestamp	No	UTC reading timestamp from the sensor device
sensor_id	String	No	Physical sensor identifier
batch_id	String	No	Batch being monitored — FK to dim_batch_master
location	String	Yes	Warehouse or vehicle zone label
temperature_c	Decimal	No	Temperature in Celsius (normalised by Spark)
humidity_pct	Decimal	Yes	Relative humidity percentage
unit	String	No	Original unit: celsius or fahrenheit
alert_flag	Boolean	No	Set by Spark if temperature outside batch range
anomaly_flag	Boolean	No	Set by Spark if reading exceeds 3-sigma window threshold
window_avg_c	Decimal	Yes	5-minute rolling window average (Spark computed)
partition_date	Date	No	Partition column for S3 and Athena pruning

4.2 Streaming: Pharmacy Sales Transactions

Source: Pharmacy POS systems. Transport: Kinesis. Frequency: Real-time on every sale.

Field	Type	Nullable	Description
transaction_id	String	No	POS transaction identifier — deduplication key
event_time	Timestamp	No	UTC sale timestamp
pharmacy_id	String	No	Pharmacy outlet identifier — FK to dim_pharmacy
product_sku	String	No	Product sold — FK to dim_product
batch_id	String	No	Batch dispensed — FK to dim_batch_master
prescription_no	String	Yes	Rx number — null for over-the-counter products
quantity_sold	Integer	No	Number of units sold
unit_price_inr	Decimal	No	Selling price per unit in INR
discount_pct	Decimal	Yes	Discount percentage applied (default 0)
revenue_inr	Decimal	No	Net revenue after discount
controlled_flag	Boolean	No	Enriched by Spark from dim_product broadcast join
product_category	String	Yes	Enriched by Spark from dim_product broadcast join
fulfillment_status	String	No	PENDING / IN_TRANSIT / COMPLETED
partition_date	Date	No	Partition column for S3 and Athena pruning

4.3 Streaming: Distributor Inventory Change Events

Source: Distributor WMS systems. Transport: Kinesis. Frequency: Real-time on every stock movement.

Field	Type	Nullable	Description
event_id	String	No	Unique movement event identifier
event_time	Timestamp	No	UTC movement timestamp
distributor_id	String	No	Distributor identifier — FK to dim_distributor
product_sku	String	No	Product moved — FK to dim_product
batch_id	String	Yes	Batch reference if tracked at this level
change_type	String	No	RECEIPT / DISPATCH / ADJUSTMENT / RETURN
quantity_delta	Integer	No	Positive for stock in, negative for stock out
current_stock	Integer	No	Stock level after this movement
reorder_point	Integer	Yes	Minimum reorder threshold for this product
below_reorder	Boolean	No	Computed by Spark: current_stock less than reorder_point
days_cover	Decimal	Yes	Computed by Spark: stock divided by 7-day avg daily sales
distributor_region	String	Yes	Enriched by Spark from dim_distributor broadcast join
partition_date	Date	No	Partition column for S3 and Athena pruning

4.4 Batch: Manufacturing Batch Master

Source: Manufacturing ERP. Transport: JDBC to S3, processed by EMR/Hive. Frequency: Daily full extract at 02:00 IST.

Field	Type	Nullable	Description
batch_id	String	No	Unique batch identifier — natural key for all joins
product_sku	String	No	Product in this batch — FK to dim_product
plant_id	String	No	Manufacturing plant — FK to dim_plant
manufacture_date	Date	No	Date production was completed
expiry_date	Date	No	Batch expiry date
batch_qty	Integer	No	Total units produced
min_temp_c	Decimal	Yes	Cold chain minimum storage temperature allowed
max_temp_c	Decimal	Yes	Cold chain maximum storage temperature allowed
batch_status	String	No	RELEASED / QUARANTINE / RECALLED / EXPIRED
release_date	Date	Yes	QA release date — null if not yet released
recall_date	Date	Yes	Date recalled — null if not recalled
recall_reason	String	Yes	Description of recall reason

4.5 Batch: Daily Inventory Snapshot

Source: Distributor ERP / WMS. Transport: JDBC to S3. Frequency: Daily closing balances at 03:00 IST.

Field	Type	Nullable	Description
snapshot_date	Date	No	Date of this closing balance snapshot
distributor_id	String	No	Distributor — FK to dim_distributor
product_sku	String	No	Product — FK to dim_product
batch_id	String	Yes	Batch tracked at this level if available
opening_stock	Integer	No	Stock at start of day
receipts	Integer	No	Units received during the day
dispatches	Integer	No	Units dispatched during the day
adjustments	Integer	No	Write-offs and corrections
returns	Integer	No	Units returned from pharmacies
closing_stock	Integer	No	Calculated: opening plus receipts minus dispatches plus returns plus adjustments
reorder_point	Integer	Yes	Minimum reorder threshold

4.6 Dimension: Product Master

Source: Product MDM system. Frequency: Daily full refresh at 01:00 IST.

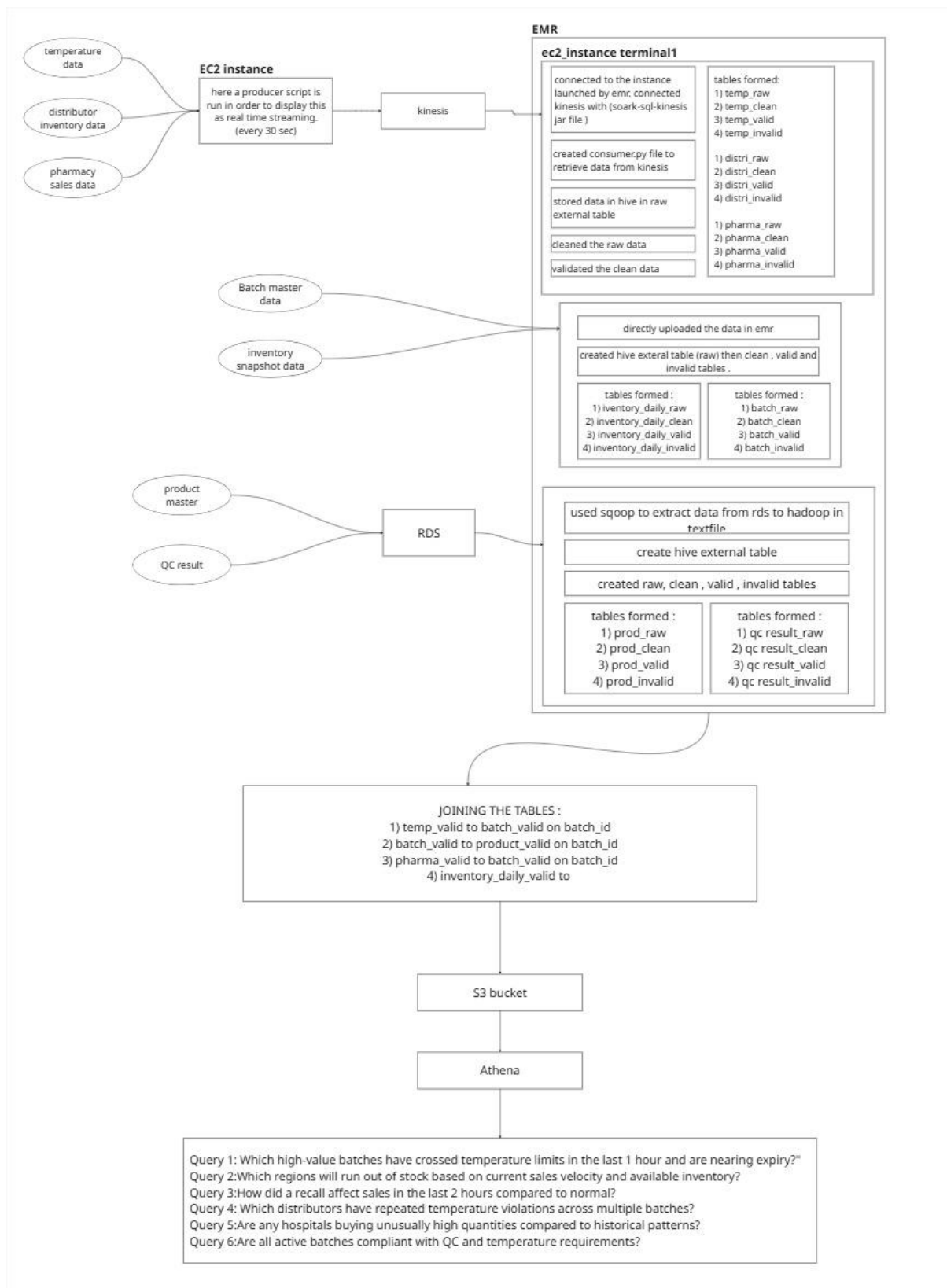
Field	Type	Nullable	Description
product_sku	String	No	Unique product code — natural key
product_name	String	No	Full commercial name
generic_name	String	Yes	INN or generic name
category	String	Yes	Antibiotic / Vaccine / Analgesic / Cardiac etc.
dosage_form	String	Yes	Tablet / Capsule / Injection / Syrup
strength	String	Yes	e.g. 500mg, 1g per 5ml
schedule	String	Yes	India drug schedule: H / H1 / G / X
controlled_flag	Boolean	No	True for narcotic or psychotropic substances
cold_chain_required	Boolean	No	True if 2 to 8 degree Celsius storage required
min_temp_c	Decimal	Yes	Minimum storage temperature (cold chain products)
max_temp_c	Decimal	Yes	Maximum storage temperature (cold chain products)
reorder_point_units	Integer	Yes	Default reorder threshold in units
unit_cost_inr	Decimal	Yes	Manufacturer cost price in INR
unit_price_mrp_inr	Decimal	Yes	Maximum retail price in INR
is_active	Boolean	No	False for discontinued products

4.7 Dimension: QC Test Results

Source: Laboratory Information Management System (LIMS). Frequency: Daily incremental append.

Field	Type	Nullable	Description
qc_id	String	No	Unique QC test record identifier
batch_id	String	No	Batch tested — FK to dim_batch_master
product_sku	String	No	Product — FK to dim_product
test_date	Date	No	Date test was conducted
lab_id	String	Yes	Laboratory that ran the test
potency_pct	Decimal	Yes	Percentage of labelled strength found
dissolution_pct	Decimal	Yes	Dissolution test result percentage
sterility_result	String	Yes	PASS / FAIL / NA
endotoxin_result	String	Yes	PASS / FAIL / NA
overall_result	String	No	PASS or FAIL
failure_reason	String	Yes	Description of failure — null if PASS
retested_flag	Boolean	No	True if this record is a retest
retest_result	String	Yes	PASS or FAIL result of the retest
approved_by	String	Yes	QA officer who approved the release

The complete implementation of the pipeline :



The screenshots of the implementation of the pipeline:

This is the version 2 of the pharma project :

We are starting from the kinesis step :

The screenshot shows the 'Create data stream' page in the AWS Management Console. The page is titled 'Create data stream' and has a sub-header 'Data stream configuration'. Under 'Data stream name', the text 'pharma_kinesis_kritika' is entered in a text box. Below this, a note states: 'Acceptable characters are uppercase and lowercase letters, numbers, underscores, hyphens and periods.' The next section is 'Data stream capacity', which has two options: 'On-demand' (selected with a radio button) and 'Provisioned' (unselected). The 'On-demand' option includes a description: 'Use this mode when your data stream's throughput requirements are unpredictable and variable. With on-demand mode, your data stream's capacity scales automatically. You can proactively scale the data stream's capacity by specifying warm throughput.' The 'Provisioned' option includes a description: 'Use provisioned mode when you can reliably estimate throughput requirements of your data stream. With provisioned mode, your data stream's capacity is fixed.' Below these options, a note states: 'On-demand mode dynamically scales Amazon Kinesis Data Streams based on your throughput needs, with pay-per-use pricing. For details on limits, visit [Quotas and limits](#). For details on pricing, visit [Kinesis pricing for on-demand mode](#).' The page footer shows the AWS logo, 'CloudShell', 'Feedback', 'Console Mobile App', and copyright information for Amazon Web Services, Inc. or its affiliates.

Keep rest as default : and create

Create an ec2 instance through which streaming will be produced:

The screenshot shows the 'Instance summary' page for an EC2 instance named 'i-0ba1cd32aa683f7dc' (kritika_pharma_basic_instance). The page displays various instance details in a grid layout. The 'Instance ID' is 'i-0ba1cd32aa683f7dc'. The 'Public IPv4 address' is '3.111.188.123'. The 'Instance state' is 'Running'. The 'Private IPv4 addresses' are '172.31.41.50'. The 'Public DNS' is 'ec2-3-111-188-123.ap-south-1.compute.amazonaws.com'. The 'Private IP DNS name (IPv4 only)' is 'ip-172-31-41-50.ap-south-1.compute.internal'. The 'Instance type' is 't3.micro'. The 'VPC ID' is 'vpc-0785f5b6c0922abd2'. The 'Auto-assigned IP address' is '3.111.188.123 [Public IP]'. The 'AWS Compute Optimizer finding' shows a warning: 'User: am:aws:iam::776137447724:user/user10 is not authorized to perform: compute-optimizer:GetEnrollmentStatus on resource: * because no identity-ba'. Below the instance summary, a terminal window shows the command 'ssh -i "Kritika_ec2.pem" ec2-user@ec2-3-111-188-123.ap-south-1.compute.amazonaws.com' and the output of the SSH session, which shows the Amazon Linux 2023 logo and the URL 'https://aws.amazon.com/linux/amazon-linux-2023'. The terminal prompt is '[ec2-user@ip-172-31-41-50 ~]\$'.

Load the data in the ec2 :

```
PS C:\Users\kritika.nimje\Downloads> scp -i "Kritika_ec2.pem" \
>> pharma_data/inventory_events.csv \
>> pharma_data/sales_transactions.csv \
>> pharma_data/temperature_events.csv \
>> ec2-user@3.111.188.123:/home/ec2-user/kinesis-project/
The authenticity of host '3.111.188.123 (3.111.188.123)' can't be established.
ED25519 key fingerprint is SHA256:wopT0eo+CxIOhR6rQx3pmXrkRRAgf4/z17LLGrQ0R/Y.
This host key is known by the following other names/addresses:
C:\Users\kritika.nimje\.ssh\known_hosts:89: ec2-3-111-188-123.ap-south-1.compute.amazonaws.com
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Warning: Permanently added '3.111.188.123' (ED25519) to the list of known hosts.
inventory_events.csv          100% 13KB 427.2KB/s 00:00
sales_transactions.csv       100% 20KB 624.5KB/s 00:00
temperature_events.csv       100% 15KB 792.9KB/s 00:00
PS C:\Users\kritika.nimje\Downloads>
```

```
[ec2-user@ip-172-31-27-188 ~]$ sudo yum install python3 python3-pip -y
```

```
[ec2-user@ip-172-31-27-188 ~]$ pip install boto3 pandas
```

```
[ec2-user@ip-172-31-41-50 ~]$ aws configure
AWS Access Key ID [None]: AKIA3JNLFTWFXFDRNFM
AWS Secret Access Key [None]: 7KPnKDzOtQRHLijtrUKH6H1BjniQaRgMMXXXR6A9
Default region name [None]: ap-south-1
Default output format [None]: json
```

```
[ec2-user@ip-172-31-41-50 ~]$ vi test.py
[ec2-user@ip-172-31-41-50 ~]$ python3 test.py
```

In the test.py file write :

```
import boto3
```

```
kinesis = boto3.client("kinesis", region_name="ap-south-1")
```

```
response = kinesis.list_streams()
```

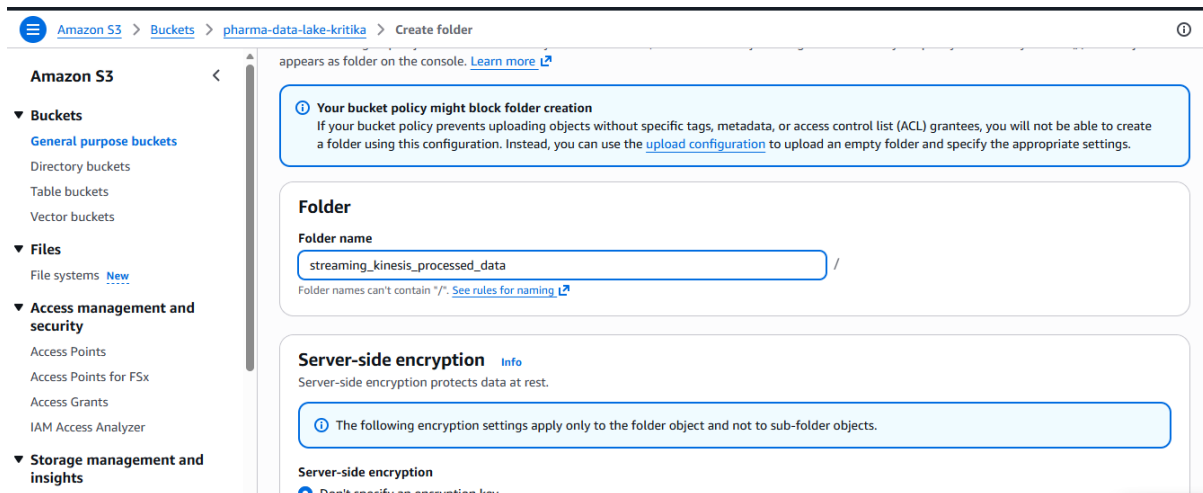
```
print(response)
```

then check the logs if it run successfully then we have the list kinesis permissions

create a directory

```
[ec2-user@ip-172-31-41-50 ~]$ mkdir kinesis-project
mkdir: cannot create directory 'kinesis-project': File exists
[ec2-user@ip-172-31-41-50 ~]$ ls
kinesis-project  test.py
[ec2-user@ip-172-31-41-50 ~]$ cd kinesis-project
[ec2-user@ip-172-31-41-50 kinesis-project]$ ls
inventory_events.csv  sales_transactions.csv  temperature_events.csv
[ec2-user@ip-172-31-41-50 kinesis-project]$ |
```

```
[ec2-user@ip-172-31-27-188 kinesis-project]$ vi producer.py
[ec2-user@ip-172-31-27-188 kinesis-project]$ python3 producer.py
```

```
[ec2-user@ip-172-31-36-8 ~]$ mkdir kinesis-project
[ec2-user@ip-172-31-36-8 ~]$ cd kinesis-project
[ec2-user@ip-172-31-36-8 kinesis-project]$
```

```
[ec2-user@ip-172-31-27-188 kinesis-project]$ vi producer.py
```

```
import boto3
```

```
import pandas as pd
```

```
import json
```

```
import time
```

```
import random
```

```
STREAM_NAME = "retail-stream-kritika"
```

```
REGION = "ap-south-1"
```

```
kinesis = boto3.client("kinesis", region_name=REGION)
```

```
# Load datasets
```

```
inventory_df = pd.read_csv("inventory_events.csv")
```

```
sales_df = pd.read_csv("sales_transactions.csv")
```

```
temperature_df = pd.read_csv("temperature_events.csv")
```

```

# Add source type
inventory_df["source_type"] = "inventory"
sales_df["source_type"] = "sales"
temperature_df["source_type"] = "temperature"

# Combine all datasets
combined_df = pd.concat([inventory_df, sales_df, temperature_df], ignore_index=True)

# Shuffle data (important)
combined_df = combined_df.sample(frac=1).reset_index(drop=True)

print("🚀 Starting streaming...")

for index, row in combined_df.iterrows():

    data = row.to_dict()

    # Convert to string for JSON
    data = {k: str(v) for k, v in data.items()}

    print(f"Sending record {index} from {data['source_type']}")

    kinesis.put_record(
        StreamName=STREAM_NAME,
        Data=json.dumps(data),
        PartitionKey=str(random.randint(1, 100))
    )

```

```
# 30 sec delay
```

```
time.sleep(30)
```

Open the terminal of emr : Hadoop

```
[hadoop@ip-172-31-27-188 ~]$ vi consumer.py|
```

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import *
```

```
from pyspark.sql.types import *
```

```
# -----
```

```
# SPARK SESSION
```

```
# -----
```

```
spark = SparkSession.builder \
```

```
    .appName("RetailStreamingConsumer") \
```

```
    .getOrCreate()
```

```
spark.sparkContext.setLogLevel("WARN")
```

```
# -----
```

```
# KINESIS CONFIG
```

```
# -----
```

```
STREAM_NAME = "pharma_kinesis_kritika"
```

```
REGION = "ap-south-1"
```

```
ENDPOINT = "https://kinesis.ap-south-1.amazonaws.com"
```

```

# -----
# READ FROM KINESIS (FIXED)
# -----

raw_df = spark.readStream \

    .format("aws-kinesis") \

    .option("kinesis.streamName", STREAM_NAME) \

    .option("kinesis.region", REGION) \

    .option("kinesis.endpointUrl", ENDPOINT) \

    .option("kinesis.startingPosition", "LATEST") \

    .load()


# Convert binary to string

json_df = raw_df.selectExpr("CAST(data AS STRING) as json_data")


# -----
# SCHEMA (SUPERSET)
# -----

schema = StructType([

    StructField("source_type", StringType()),

    # inventory

    StructField("event_id", StringType()),

    StructField("event_time", StringType()),

    StructField("distributor_id", StringType()),

    StructField("product_sku", StringType()),

    StructField("batch_id", StringType()),

    StructField("change_type", StringType()),

    StructField("quantity_delta", StringType()),

```

```

StructField("current_stock", StringType()),

# sales
StructField("transaction_id", StringType()),
StructField("pharmacy_id", StringType()),
StructField("quantity_sold", StringType()),
StructField("unit_price", StringType()),
StructField("discount_pct", StringType()),
StructField("revenue", StringType()),
StructField("fulfillment_status", StringType()),

# temperature
StructField("sensor_id", StringType()),
StructField("temperature", StringType()),
StructField("unit", StringType()),
StructField("humidity", StringType())
])

# -----
# PARSE JSON
# -----

parsed_df = json_df \
    .select(from_json(col("json_data"), schema).alias("data")) \
    .select("data.*")

# Convert timestamp
parsed_df = parsed_df.withColumn("event_time", to_timestamp("event_time"))

```

```

# Partition column

parsed_df = parsed_df.withColumn("partition_date", to_date("event_time"))

# -----

# SPLIT STREAMS

# -----

inventory_df = parsed_df.filter(col("source_type") == "inventory")
sales_df = parsed_df.filter(col("source_type") == "sales")
temperature_df = parsed_df.filter(col("source_type") == "temperature")

# =====

# 📦 TEMPERATURE CLEANING & VALIDATION

# =====

temp_clean = temperature_df \

    .withColumn("temperature", col("temperature").cast("double")) \

    .withColumn("humidity", col("humidity").cast("double")) \

    .withColumn(

        "temperature_c",

        when(col("unit") == "F", (col("temperature") - 32) * 5/9)

        .otherwise(col("temperature"))

    ) \

    .withColumn(

        "valid_temp",

        (col("temperature_c") > -50) & (col("temperature_c") < 100)

    ) \

    .filter(col("valid_temp") == True)

# =====

```

```
# 📌 SALES CLEANING & VALIDATION
```

```
# =====
```

```
sales_clean = sales_df \
    .withColumn("quantity_sold", col("quantity_sold").cast("int")) \
    .withColumn("unit_price", col("unit_price").cast("double")) \
    .withColumn("discount_pct", col("discount_pct").cast("double")) \
    .withColumn("revenue", col("revenue").cast("double")) \
    .withColumn(
        "valid_sale",
        (col("quantity_sold") > 0) & (col("unit_price") > 0)
    ) \
    .filter(col("valid_sale") == True)
```

```
# =====
```

```
# 📦 INVENTORY CLEANING & VALIDATION
```

```
# =====
```

```
inventory_clean = inventory_df \
    .withColumn("quantity_delta", col("quantity_delta").cast("int")) \
    .withColumn("current_stock", col("current_stock").cast("int")) \
    .withColumn(
        "valid_change",
        col("change_type").isin("RECEIPT", "DISPATCH", "RETURN", "ADJUSTMENT")
    ) \
    .filter(col("valid_change") == True)
```

```
# -----
```

```
# S3 CONFIG
```

```
# -----
```

```
S3_BASE = "s3://pharma-data-lake-kritika/streaming_kinesis_processed_data/"
```

```
# =====
```

```
# WRITE STREAMS (MICRO-BATCH: 30 sec)
```

```
# =====
```

```
# Temperature
```

```
temp_query = temp_clean.writeStream \
```

```
    .format("parquet") \
```

```
    .option("path", S3_BASE + "temperature_streaming_data/") \
```

```
    .option("checkpointLocation", S3_BASE + "checkpoints/temperature/") \
```

```
    .partitionBy("partition_date") \
```

```
    .outputMode("append") \
```

```
    .trigger(processingTime="120 seconds") \
```

```
    .start()
```

```
# Sales
```

```
sales_query = sales_clean.writeStream \
```

```
    .format("parquet") \
```

```
    .option("path", S3_BASE + "sales_streaming_data/") \
```

```
    .option("checkpointLocation", S3_BASE + "checkpoints/sales/") \
```

```
    .partitionBy("partition_date") \
```

```
    .outputMode("append") \
```

```
    .trigger(processingTime="120 seconds") \
```

```
    .start()
```

```
# Inventory
```

```
inventory_query = inventory_clean.writeStream \
```

```
.format("parquet") \
.option("path", S3_BASE + "inventory_streaming_data/") \
.option("checkpointLocation", S3_BASE + "checkpoints/inventory/") \
.partitionBy("partition_date") \
.outputMode("append") \
.trigger(processingTime="120 seconds") \
.start()
```

```
# -----
```

```
# KEEP STREAM RUNNING
```

```
# -----
```

```
spark.streams.awaitAnyTermination()
```

Start the producer :

```
[ec2-user@ip-172-31-27-188 kinesis-project]$ python3 producer.py|
```

Start the consumer:

```
[hadoop@ip-172-31-27-188 ~]$ spark-submit --jars spark-sql-kinesis_2.12-1.2.0_spark-3.0.jar consumer.py
```

Keep both on side by side

```
ec2-user@ip-172-31-27-188:~$ vi producer.py
[ec2-user@ip-172-31-27-188 kinesis-project]$ python3 producer.py
/home/ec2-user/.local/lib/python3.7/site-packages/boto3/compat.py:82: PythonDeprecationWarning: Boto3 will no longer support Python 3.7 starting
December 13, 2023. To continue receiving service updates, bug fixes, and security updates please upgrade to Python 3.8 or later. More information can be found here: https://aws.amazon.com/blogs/developer/python-support-policy-updates-for-aws-sdks-and-tools/
  warnings.warn(warning, PythonDeprecationWarning)
Starting streaming...
Sending record 0 from temperature
Sending record 1 from temperature
Sending record 2 from sales
Sending record 3 from temperature
Sending record 4 from sales
Sending record 5 from temperature
Sending record 6 from inventory
Sending record 7 from inventory
Sending record 8 from temperature
Sending record 9 from temperature
Sending record 10 from inventory

hadoop@ip-172-31-27-188:~$
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /executors/json: org
.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /executors/threadDum
p: org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /executors/threadDum
p/json: org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /static: org.apache
.hadoop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /: org.apache.hadoop
.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /api: org.apache.had
oop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /jobs/job/kill: org.
apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /stages/stage/kill:
org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO ServerInfo: Adding filter to /metrics/json: org.a
pache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter
26/04/10 08:26:21 INFO YarnClientSchedulerBackend: SchedulerBackend is r
eady for scheduling beginning after reached minRegisteredResourcesRatio:
0.0
26/04/10 08:26:29 WARN ResolveWriteToStream: spark.sql.adaptive.enabled
is not supported in streaming DataFrames/Datasets and will be disabled.
26/04/10 08:26:30 WARN ResolveWriteToStream: spark.sql.adaptive.enabled
is not supported in streaming DataFrames/Datasets and will be disabled.
26/04/10 08:26:30 WARN ResolveWriteToStream: spark.sql.adaptive.enabled
is not supported in streaming DataFrames/Datasets and will be disabled.
26/04/10 08:27:10 WARN ProcessingTimeExecutor: Current batch is falling
behind. The trigger interval is 30000 milliseconds, but spent 39714 mill
iseconds
26/04/10 08:27:36 WARN ProcessingTimeExecutor: Current batch is falling
behind. The trigger interval is 30000 milliseconds, but spent 36948 mill
iseconds
```

Now updating the pipeline and adding the hive tables

Updated consumer.py

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import *
```

```
from pyspark.sql.types import *
```

```
# -----
```

```
# SPARK SESSION WITH HIVE
```

```
# -----
```

```
spark = SparkSession.builder \
```

```
    .appName("RetailStreamingConsumer") \
```

```
    .enableHiveSupport() \
```

```
    .config("spark.sql.shuffle.partitions", "50") \
```

```
    .getOrCreate()
```

```
spark.sparkContext.setLogLevel("WARN")
```

```

# -----
# KINESIS CONFIG
# -----

STREAM_NAME = "pharma_kinesis_kritika"

REGION = "ap-south-1"

ENDPOINT = "https://kinesis.ap-south-1.amazonaws.com"


# -----
# READ FROM KINESIS
# -----

raw_df = spark.readStream \

    .format("aws-kinesis") \

    .option("kinesis.streamName", STREAM_NAME) \

    .option("kinesis.region", REGION) \

    .option("kinesis.endpointUrl", ENDPOINT) \

    .option("kinesis.startingPosition", "LATEST") \

    .load()


# Convert binary to string

json_df = raw_df.selectExpr("CAST(data AS STRING) as json_data")


# -----
# SCHEMA
# -----

schema = StructType([

    StructField("source_type", StringType()),

    # inventory

```

```
StructField("event_id", StringType()),
StructField("event_time", StringType()),
StructField("distributor_id", StringType()),
StructField("product_sku", StringType()),
StructField("batch_id", StringType()),
StructField("change_type", StringType()),
StructField("quantity_delta", StringType()),
StructField("current_stock", StringType()),
```

```
# sales
```

```
StructField("transaction_id", StringType()),
StructField("pharmacy_id", StringType()),
StructField("quantity_sold", StringType()),
StructField("unit_price", StringType()),
StructField("discount_pct", StringType()),
StructField("revenue", StringType()),
StructField("fulfillment_status", StringType()),
```

```
# temperature
```

```
StructField("sensor_id", StringType()),
StructField("temperature", StringType()),
StructField("unit", StringType()),
StructField("humidity", StringType())
```

```
])
```

```
# -----
```

```
# PARSE JSON
```

```
# -----
```

```

parsed_df = json_df \

    .select(from_json(col("json_data"), schema).alias("data")) \

    .select("data.*")


# Convert timestamp
parsed_df = parsed_df.withColumn("event_time", to_timestamp("event_time"))


# Partition column
parsed_df = parsed_df.withColumn("partition_date", to_date("event_time"))


# -----
# SPLIT STREAMS
# -----

inventory_df = parsed_df.filter(col("source_type") == "inventory")
sales_df = parsed_df.filter(col("source_type") == "sales")
temperature_df = parsed_df.filter(col("source_type") == "temperature")


# =====
# TEMPERATURE CLEANING
# =====

temp_clean = temperature_df \

    .withColumn("temperature", col("temperature").cast("double")) \

    .withColumn("humidity", col("humidity").cast("double")) \

    .withColumn(

        "temperature_c",

        when(col("unit") == "F", (col("temperature") - 32) * 5/9)

        .otherwise(col("temperature"))

    ) \

```

```

.filter((col("temperature_c") > -50) & (col("temperature_c") < 100)) \

.select("sensor_id", "temperature", "humidity", "temperature_c", "unit", "event_time",
"partition_date")

# =====

# SALES CLEANING

# =====

sales_clean = sales_df \

.withColumn("quantity_sold", col("quantity_sold").cast("int")) \

.withColumn("unit_price", col("unit_price").cast("double")) \

.withColumn("discount_pct", col("discount_pct").cast("double")) \

.withColumn("revenue", col("revenue").cast("double")) \

.filter((col("quantity_sold") > 0) & (col("unit_price") > 0)) \

.select("transaction_id", "pharmacy_id", "quantity_sold", "unit_price", "discount_pct",
"revenue", "fulfillment_status", "event_time", "partition_date")

# =====

# INVENTORY CLEANING

# =====

inventory_clean = inventory_df \

.withColumn("quantity_delta", col("quantity_delta").cast("int")) \

.withColumn("current_stock", col("current_stock").cast("int")) \

.filter(col("change_type").isin("RECEIPT", "DISPATCH", "RETURN", "ADJUSTMENT")) \

.select("event_id", "distributor_id", "product_sku", "batch_id", "change_type",
"quantity_delta", "current_stock", "event_time", "partition_date")

# =====

# WRITE TO HIVE TABLES (MICRO-BATCH)

# =====

```

Temperature

```
temp_query = temp_clean.writeStream \  
  .outputMode("append") \  
  .format("parquet") \  
  .option("checkpointLocation", "s3://pharma-data-lake-  
kritika/checkpoints/temperature/") \  
  .partitionBy("partition_date") \  
  .trigger(processingTime="120 seconds") \  
  .toTable("pharma_streaming_db.temperature_streaming")
```

Sales

```
sales_query = sales_clean.writeStream \  
  .outputMode("append") \  
  .format("parquet") \  
  .option("checkpointLocation", "s3://pharma-data-lake-kritika/checkpoints/sales/") \  
  .partitionBy("partition_date") \  
  .trigger(processingTime="120 seconds") \  
  .toTable("pharma_streaming_db.sales_streaming")
```

Inventory

```
inventory_query = inventory_clean.writeStream \  
  .outputMode("append") \  
  .format("parquet") \  
  .option("checkpointLocation", "s3://pharma-data-lake-  
kritika/checkpoints/inventory/") \  
  .partitionBy("partition_date") \  
  .trigger(processingTime="120 seconds") \  
  .toTable("pharma_streaming_db.inventory_streaming")
```

```
# -----
```

```
# KEEP STREAM RUNNING
```

```
# -----
```

```
spark.streams.awaitAnyTermination()
```

create the spark database

```
[hadoop@ip-172-31-27-188 ~]$ spark-sql
```

```
hive> exit;
```

```
[hadoop@ip-172-31-27-188 ~]$ spark-sql
```

```
Setting default log level to "WARN".
```

```
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
```

```
26/04/10 09:33:21 WARN HiveConf: HiveConf of name hive.server2.thrift.url does not exist
```

```
26/04/10 09:33:23 WARN Client: Neither spark.yarn.jars nor spark.yarn.archive is set, falling back to uploading libraries under SPARK_HOME.
```

```
Spark master: yarn, Application Id: application_1775799336076_0021
```

```
spark-sql (default)> |
```

```
spark-sql (default)> CREATE DATABASE IF NOT EXISTS pharma_streaming_db;
```

```
Time taken: 2.593 seconds
```

```
spark-sql (default)> |
```

```
spark-sql (default)> use pharma_streaming_db;
```

```
Time taken: 0.001 seconds
```

```
spark-sql (pharma_streaming_db)> CREATE TABLE pharma_streaming_db.temperature_streaming (
```

```
>     sensor_id STRING,  
>     temperature DOUBLE,  
>     humidity DOUBLE,  
>     temperature_c DOUBLE,  
>     unit STRING,  
>     event_time TIMESTAMP,  
>     partition_date DATE  
> )
```

```
> USING PARQUET
```

```
> PARTITIONED BY (partition_date)
```

```
> LOCATION 's3://pharma-data-lake-kritika
```

```
/streaming_kinesis/temperature';
```

```
Time taken: 0.216 seconds
```

```
spark-sql (pharma_streaming_db)> |
```

```
spark-sql (pharma_streaming_db)> CREATE TABLE pharma_streaming_db.sales_
streaming (
    >     transaction_id STRING,
    >     pharmacy_id STRING,
    >     quantity_sold INT,
    >     unit_price DOUBLE,
    >     discount_pct DOUBLE,
    >     revenue DOUBLE,
    >     fulfillment_status STRING,
    >     event_time TIMESTAMP,
    >     partition_date DATE
    > )
    > USING PARQUET
    > PARTITIONED BY (partition_date)
    > LOCATION 's3://pharma-data-lake-kritika
/streaming_kinesis/sales';
Time taken: 0.24 seconds
spark-sql (pharma_streaming_db)> |
```

```
spark-sql (pharma_streaming_db)> CREATE TABLE pharma_streaming_db.invent
ory_streaming (
    >     event_id STRING,
    >     distributor_id STRING,
    >     product_sku STRING,
    >     batch_id STRING,
    >     change_type STRING,
    >     quantity_delta INT,
    >     current_stock INT,
    >     event_time TIMESTAMP,
    >     partition_date DATE
    > )
    > USING PARQUET
    > PARTITIONED BY (partition_date)
    > LOCATION 's3://pharma-data-lake-kritika
/streaming_kinesis/inventory';
Time taken: 0.213 seconds
spark-sql (pharma_streaming_db)> |
```

```
spark-sql (default)> show databases;
default
pharma_streaming_db
Time taken: 3.023 seconds, Fetched 2 row(s)
```

```
spark-sql (default)> use pharma_streaming_db;
Time taken: 0.049 seconds
```

```

spark-sql (pharma_streaming_db)> select * from inventory_streaming;
IE128  D07      SKU014  B002  DISPATCH      -16    13    2026-03-
27 17:32:57.020755      2026-03-27
IE088  D07      SKU009  B010  RECEIPT 94      414    2026-03-26 17:32
:57.020358      2026-03-26
IE184  D07      SKU024  B009  RETURN 41      430    2026-03-16 17:32
:57.02117      2026-03-16
IE133  D05      SKU012  B006  RECEIPT 47      418    2026-04-08 17:32
:57.020835      2026-04-08
IE010  D08      SKU000  B043  RETURN 44      -12    2026-03-29 17:32
:57.019639      2026-03-29
IE104  D00      SKU015  B022  DISPATCH      59    299    2026-04-
04 17:32:57.020487      2026-04-04
Time taken: 12.501 seconds, Fetched 6 row(s)
spark-sql (pharma_streaming_db)> |

```

The screenshot shows a terminal window with two panes. The left pane displays a list of records being sent from various sources (sales, temperature, inventory) to a streaming database. The right pane shows the execution of SQL queries in a Spark shell, including showing databases, using the 'pharma_streaming_db' database, showing tables, and querying the 'inventory_streaming' table. The output of the query matches the data shown in the first image.

```

ec2-user@ip-172-31-27-188:~$ cat producer.py
Sending record 143 from sales
Sending record 144 from temperature
Sending record 145 from inventory
Sending record 146 from inventory
Sending record 147 from inventory
Sending record 148 from sales
Sending record 149 from sales
Sending record 150 from temperature
Sending record 151 from temperature
Sending record 152 from inventory
Sending record 153 from temperature
Sending record 154 from temperature
Sending record 155 from inventory
Sending record 156 from temperature
Sending record 157 from sales
Sending record 158 from inventory
Sending record 159 from inventory
Sending record 160 from inventory
Sending record 161 from temperature
Sending record 162 from sales
Sending record 163 from temperature
Sending record 164 from temperature
Sending record 165 from sales
Sending record 166 from sales
Sending record 167 from sales
Sending record 168 from temperature
Sending record 169 from sales
Sending record 170 from temperature
Sending record 171 from sales
^CTraceback (most recent call last):
  File "producer.py", line 46, in <module>
    time.sleep(5)
KeyboardInterrupt
[ec2-user@ip-172-31-27-188 kinesio-project]$

hadoop@ip-172-31-27-188:~$ spark-shell
is not supported in streaming DataFrames/Datasets and will be disabled.
Spark master: yarn, Application Id: application_1775799336076_0024
spark-sql (default)> show databases;
default
pharma_streaming_db
Time taken: 3.023 seconds, Fetched 2 row(s)
spark-sql (default)> show databases;
default
pharma_streaming_db
Time taken: 0.056 seconds, Fetched 2 row(s)
spark-sql (default)> use pharma_streaming_db;
Time taken: 0.049 seconds
spark-sql (pharma_streaming_db)> show tables;
inventory_streaming
sales_streaming
temperature_streaming
Time taken: 0.138 seconds, Fetched 3 row(s)
spark-sql (pharma_streaming_db)> select * from inventory_streaming;
IE128  D07      SKU014  B002  DISPATCH      -16    13    2026-03-
27 17:32:57.020755      2026-03-27
IE088  D07      SKU009  B010  RECEIPT 94      414    2026-03-26 17:32
:57.020358      2026-03-26
IE184  D07      SKU024  B009  RETURN 41      430    2026-03-16 17:32
:57.02117      2026-03-16
IE133  D05      SKU012  B006  RECEIPT 47      418    2026-04-08 17:32
:57.020835      2026-04-08
IE010  D08      SKU000  B043  RETURN 44      -12    2026-03-29 17:32
:57.019639      2026-03-29
IE104  D00      SKU015  B022  DISPATCH      59    299    2026-04-
04 17:32:57.020487      2026-04-04
Time taken: 12.501 seconds, Fetched 6 row(s)
spark-sql (pharma_streaming_db)> exit

```

Successfully streamed and got stored in the tables

The data is been successfully stored in the bucket :

Amazon S3 > Buckets > pharma-data-lake-kritika > streaming_kinesis/

Objects Properties

Objects (6)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	inventory_\$folder\$	-	April 10, 2026, 14:35:55 (UTC+05:30)	0 B	Standard
☐	inventory/	Folder	-	-	-
☐	sales_\$folder\$	-	April 10, 2026, 14:35:44 (UTC+05:30)	0 B	Standard
☐	sales/	Folder	-	-	-
☐	temperature_\$folder\$	-	April 10, 2026, 14:35:32 (UTC+05:30)	0 B	Standard
☐	temperature/	Folder	-	-	-

Amazon S3 > Buckets > pharma-data-lake-kritika > streaming_kinesis/ > inventory/

Objects Properties

Objects (47)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	_spark_metadata_\$folder\$	-	April 10, 2026, 15:22:07 (UTC+05:30)	0 B	Standard
☐	_spark_metadata/	Folder	-	-	-
☐	part-00000-115e42d2-cfc5-459c-ac03-a49a7f7931a1-c000.snappy.parquet	parquet	April 10, 2026, 15:33:23 (UTC+05:30)	2.5 KB	Standard
☐	part-00000-204f0e41-b722-4d43-9662-da7a7ad4b858-c000.snappy.parquet	parquet	April 10, 2026, 15:22:26 (UTC+05:30)	1.1 KB	Standard
☐	part-00000-45cbf662-36df-4e27-b270-3af7acd0438-c000.snappy.parquet	parquet	April 10, 2026, 15:34:18 (UTC+05:30)	1.1 KB	Standard

Amazon S3 > Buckets > pharma-data-lake-kritika > streaming_kinesis/ > sales/

sales/

Objects Properties

Objects (44)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	_spark_metadata_\$folder\$	-	April 10, 2026, 15:22:06 (UTC+05:30)	0 B	Standard
☐	_spark_metadata/	Folder	-	-	-
☐	part-00000-190843b7-d581-4401-8f55-09c25f89eab3-c000.snappy.parquet	parquet	April 10, 2026, 15:35:19 (UTC+05:30)	1.1 KB	Standard
☐	part-00000-1e3f71d1-bca1-4873-8b80-5b51b5c6c9e9-c000.snappy.parquet	parquet	April 10, 2026, 15:24:07 (UTC+05:30)	2.6 KB	Standard

Completing the rds to emr pipeline again : most things in main docu

Upload the jar file on the ec2 instance :

```
PS C:\Users\kritika.nimje> cd Downloads
PS C:\Users\kritika.nimje\Downloads> scp -i "Kritika_ec2.pem" mysql-connector-java-8.0.22.jar hadoop@3.108.3.204:/home/hadoop/
mysql-connector-java-8.0.22.jar
PS C:\Users\kritika.nimje\Downloads>
```

```
[hadoop@ip-172-31-27-188 ~]$ sudo mv ~/mysql-connector-java-8.0.22.jar /usr/lib/sqoop/lib/
```

```
[hadoop@ip-172-31-27-188 ~]$ sudo mv ~/mysql-connector-java-8.0.22.jar /usr/lib/sqoop/lib/
[hadoop@ip-172-31-27-188 ~]$ sudo chown root:root /usr/lib/sqoop/lib/mysql-connector-java-8.0.22.jar
[hadoop@ip-172-31-27-188 ~]$ sudo chmod 644 /usr/lib/sqoop/lib/mysql-connector-java-8.0.22.jar
[hadoop@ip-172-31-27-188 ~]$ ls -lh /usr/lib/sqoop/lib | grep mysql
-rw-r--r-- 1 root root 2.3M Apr 10 10:52 mysql-connector-java-8.0.22.jar
[hadoop@ip-172-31-27-188 ~]$
```

Loaded both the table from rds to Hadoop in text file via sqoop:

```
[hadoop@ip-172-31-27-188 ~]$ sqoop import \
> --connect jdbc:mysql://retailpulse-kritika-db.c58qessmggt4.ap-south-1.rds.amazonaws.com:3306/pharma_kritika \
> --username admin \
> --password Admin1234 \
> --driver com.mysql.cj.jdbc.Driver \
> --table qc_results \
> --target-dir /user/hadoop/qc_results \
> --as-textfile \
> --fields-terminated-by ',' \
> --lines-terminated-by '\n' \
> --m 1
```

```
[hadoop@ip-172-31-27-188 ~]$ sqoop import \
> --connect jdbc:mysql://retailpulse-kritika-db.c58qessmggt4.ap-south-1.rds.amazonaws.com:3306/pharma_kritika \
> --username admin \
> --password Admin1234 \
> --driver com.mysql.cj.jdbc.Driver \
> --table product_master \
> --target-dir /user/hadoop/product_master \
> --as-textfile \
> --fields-terminated-by ',' \
> --lines-terminated-by '\n' \
> --m 1
```

Now lets create the schema of the data in hive

```
hive> CREATE DATABASE pharma_db_kritika;
OK
Time taken: 0.589 seconds
hive> USE pharma_db_kritika;
OK
```

```

hive> CREATE EXTERNAL TABLE product_master_raw (
>   product_sku STRING,
>   product_name STRING,
>   generic_name STRING,
>   category STRING,
>   dosage_form STRING,
>   strength STRING,
>   schedule STRING,
>   controlled_flag STRING,
>   cold_chain_required STRING,
>   min_temp_c STRING,
>   max_temp_c STRING,
>   reorder_point_units STRING,
>   unit_cost_inr STRING,
>   unit_price_mrp_inr STRING,
>   is_active STRING
> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> STORED AS TEXTFILE
> LOCATION '/user/hadoop/product_master';

```

OK

Time taken: 0.17 seconds

```
hive> SELECT * FROM product_master_raw LIMIT 10;
```

OK

SKU000	Vaccine_Drug_SKU000	Vaccine Injection	250mg	True	True	0.00	0.00	85	387.00	NULL	NULL	NULL	NULL
SKU001	Analgesic_Drug_SKU001	Analgesic Capsule	500mg	True	False	0.00	0.00	57	57.00	NULL	NULL	NULL	NULL
SKU002	Analgesic_Drug_SKU002	Analgesic Tablet	1g	False	False	10.00	5.00	106	239.00	NULL	NULL	NULL	NULL
SKU003	Antibiotic_Drug_SKU003	Antibiotic Tablet	1g	False	False	10.00	5.00	137	152.00	NULL	NULL	NULL	NULL
SKU004	Analgesic_Drug_SKU004	Analgesic Tablet	500mg	True	False	0.00	0.00	141	443.00	NULL	NULL	NULL	NULL
SKU005	Antibiotic_Drug_SKU005	Antibiotic Injection	1g	True	True	0.00	0.00	146	50.00	NULL	NULL	NULL	NULL
SKU006	Antibiotic_Drug_SKU006	Antibiotic Injection	1g	True	False	0.00	0.00	67	33.00	NULL	NULL	NULL	NULL
SKU007	Vaccine_Drug_SKU007	Vaccine Tablet	250mg	True	True	2.00	8.00	147	152.00	NULL	NULL	NULL	NULL
SKU008	Antibiotic_Drug_SKU008	Antibiotic Injection	250mg	False	False	0.00	0.00	68	321.00	NULL	NULL	NULL	NULL
SKU009	Vaccine_Drug_SKU009	Vaccine Tablet	250mg	False	True	2.00	8.00	147	148.00	NULL	NULL	NULL	NULL

Time taken: 1.242 seconds, Fetched: 10 row(s)

```

hive> DROP TABLE IF EXISTS qc_results_raw;
OK
Time taken: 0.086 seconds
hive> CREATE EXTERNAL TABLE qc_results_raw (
  >   qc_id STRING,
  >   batch_id STRING,
  >   product_sku STRING,
  >   test_datetime STRING,
  >   potency_pct STRING,
  >   overall_result STRING,
  >   failure_reason STRING
  > )
  > ROW FORMAT DELIMITED
  > FIELDS TERMINATED BY ','
  > STORED AS TEXTFILE
  > LOCATION '/user/hadoop/qc_results';
OK
Time taken: 0.058 seconds

```

Cleaning the data :

```

Time taken: 0.009 seconds
hive> CREATE TABLE qc_results_clean AS
  > SELECT
  >   qc_id,
  >   batch_id,
  >   product_sku,
  >   CAST(test_datetime AS TIMESTAMP) AS test_datetime,
  >   CAST(potency_pct AS DOUBLE) AS potency_pct,
  >   CASE
  >     WHEN UPPER(TRIM(overall_result)) = 'PASS' THEN 'PASS'
  >     WHEN UPPER(TRIM(overall_result)) = 'FAIL' THEN 'FAIL'
  >     ELSE NULL
  >   END AS overall_result,
  >   NULLIF(TRIM(failure_reason), '') AS failure_reason
  > FROM qc_results_raw;
Query ID = hadoop_20260410111442_53333409-a640-4ad4-91c0-2bb1df12a792
Total jobs = 1
Launching Job 1 out of 1
Tez session was closed. Reopening...
Session re-established.
Session re-established.
Status: Running (Executing on YARN cluster with App id application_1775799336076_0030)

Map 1: -/-
Map 1: 0/1
Map 1: 0(+1)/1
Map 1: 1/1
Moving data to directory hdfs://ip-172-31-27-188.ap-south-1.compute.internal:8020/user/hi
OK

```

```

hive> CREATE TABLE qc_results_valid AS
> SELECT *
> FROM qc_results_clean
> WHERE
>     overall_result IN ('PASS', 'FAIL')
>     AND potency_pct BETWEEN 80 AND 120
>     AND (
>         overall_result = 'PASS'
>         OR (overall_result = 'FAIL' AND failure_reason IS NOT NULL)
>     );
Query ID = hadoop_20260410111548_15e10bfc-8da6-442c-baae-8153d35c547e
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1775799336076_0030)

Map 1: -/-
Map 1: 0/1
Map 1: 0(+1)/1
Map 1: 1/1
Moving data to directory hdfs://ip-172-31-27-188.ap-south-1.compute.internal:8020/user/hive/w
OK
Time taken: 5.739 seconds
hive>

```

```

hive> CREATE TABLE qc_results_invalid AS
> SELECT *
> FROM qc_results_clean
> WHERE
>     overall_result NOT IN ('PASS', 'FAIL')
>     OR potency_pct NOT BETWEEN 80 AND 120
>     OR (
>         overall_result = 'FAIL' AND failure_reason IS NULL
>     );
Query ID = hadoop_20260410111641_997b9eb4-8d52-47f2-a8d0-2be6719c192e
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1775799336076_0030)

Map 1: -/-
Map 1: 0/1
Map 1: 0(+1)/1
Map 1: 1/1
Moving data to directory hdfs://ip-172-31-27-188.ap-south-1.compute.internal:8020/user/hi
OK
Time taken: 4.548 seconds
hive>

```

```

hive> SELECT COUNT(*) FROM qc_results_clean;
OK
200
Time taken: 0.086 seconds, Fetched: 1 row(s)
hive> SELECT COUNT(*) FROM qc_results_valid;
OK
183
Time taken: 0.061 seconds, Fetched: 1 row(s)
hive> SELECT COUNT(*) FROM qc_results_invalid;
OK
17
Time taken: 0.059 seconds, Fetched: 1 row(s)
hive>

```

```

FAILED: ParseException line 20:10 mismatched input '/' expecting StringLiteral near 'LOCATION' in table location specification
hive> CREATE EXTERNAL TABLE product_master_raw (
  >   product_sku STRING,
  >   product_name STRING,
  >   category STRING,
  >   dosage_form STRING,
  >   strength STRING,
  >   controlled_flag STRING,
  >   cold_chain_required STRING,
  >   min_temp_c STRING,
  >   max_temp_c STRING,
  >   reorder_point_units STRING,
  >   unit_price_mrp_inr STRING
  > )
  > ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
  > WITH SERDEPROPERTIES (
  >   "separatorChar" = ",",
  >   "quoteChar" = "\""
  > )
  > STORED AS TEXTFILE
  > LOCATION '/user/hadoop/product_master';
OK
Time taken: 0.032 seconds
hive> select * from product_master_raw limit 10;
OK
SKU000 Vaccine_Drug_SKU000 Vaccine Injection 250mg True True 0.00 0.00 85 387.00
SKU001 Analgesic_Drug_SKU001 Analgesic Capsule 500mg True False 0.00 0.00 57 57.00
SKU002 Analgesic_Drug_SKU002 Analgesic Tablet 1g False False 10.00 5.00 106 239.00
SKU003 Antibiotic_Drug_SKU003 Antibiotic Tablet 1g False False 10.00 5.00 137 152.00
SKU004 Analgesic_Drug_SKU004 Analgesic Tablet 500mg True False 0.00 0.00 141 443.00
SKU005 Antibiotic_Drug_SKU005 Antibiotic Injection 1g True True 0.00 0.00 146 50.00
SKU006 Antibiotic_Drug_SKU006 Antibiotic Injection 1g True False 0.00 0.00 67 33.00
SKU007 Vaccine_Drug_SKU007 Vaccine Tablet 250mg True True 2.00 8.00 147 152.00
SKU008 Antibiotic_Drug_SKU008 Antibiotic Injection 250mg False False 0.00 0.00 68 321.00
SKU009 Vaccine_Drug_SKU009 Vaccine Tablet 250mg False True 2.00 8.00 147 148.00

```

```

Time taken: 0.002 seconds, Fetched: 1 row(s)
hive> CREATE TABLE pharma_db_kritika.product_master_valid
> STORED AS PARQUET
> AS
> SELECT *
> FROM pharma_db_kritika.product_master_clean
> WHERE
>
>     /* Mandatory fields */
>     product_sku IS NOT NULL
>     AND product_name IS NOT NULL
>     AND category IS NOT NULL
>     AND dosage_form IS NOT NULL
>
>     /* Price validation */
>     AND unit_price IS NOT NULL
>     AND unit_price > 0
>
>     /* Reorder point validation */
>     AND (reorder_point_units IS NULL OR reorder_point_units >= 0)
>
>     /* Temperature validation */
>     AND (
>         (min_temp_c IS NULL AND max_temp_c IS NULL)
>         OR (min_temp_c <= max_temp_c)
>     )
>
>     /* Cold chain validation */
>     AND (
>         cold_chain_required = FALSE
>         OR (min_temp_c IS NOT NULL AND max_temp_c IS NOT NULL)
>     );
Query ID = hadoop_20260410121250_d5be8a0d-587a-4fd7-8b10-23727447ffea
Total jobs = 1

```

```

26
Time taken: 0.054 seconds, Fetched: 1 row(s)
hive> select * from product_master_valid;
OK
SKU000 Vaccine_Drug_SKU000 Vaccine Injection 250mg true true 0.0 0.0 85 387.0
SKU001 Analgesic_Drug_SKU001 Analgesic Capsule 500mg true false 0.0 0.0 57 57.0
SKU004 Analgesic_Drug_SKU004 Analgesic Tablet 500mg true false 0.0 0.0 141 443.0
SKU005 Antibiotic_Drug_SKU005 Antibiotic Injection 1g true true 0.0 0.0 146 50.0
SKU006 Antibiotic_Drug_SKU006 Antibiotic Injection 1g true false 0.0 0.0 67 33.0
SKU007 Vaccine_Drug_SKU007 Vaccine Tablet 250mg true true 2.0 8.0 147 152.0
SKU008 Antibiotic_Drug_SKU008 Antibiotic Injection 250mg false false 0.0 0.0 68 321.0
SKU009 Vaccine_Drug_SKU009 Vaccine Tablet 250mg false true 2.0 8.0 147 148.0
SKU010 Cardiac_Drug_SKU010 Cardiac Capsule 500mg true false 0.0 0.0 108 430.0
SKU011 Analgesic_Drug_SKU011 Analgesic Tablet 1g false false 0.0 0.0 104 345.0
SKU012 Antibiotic_Drug_SKU012 Antibiotic Tablet 500mg true false 0.0 0.0 113 391.0
SKU013 Antibiotic_Drug_SKU013 Antibiotic Capsule 500mg false true 2.0 8.0 106 80.0
SKU014 Antibiotic_Drug_SKU014 Antibiotic Tablet 250mg true true 0.0 0.0 158 315.0
SKU015 Analgesic_Drug_SKU015 Analgesic Capsule 500mg true false 0.0 0.0 79 359.0
SKU016 Cardiac_Drug_SKU016 Cardiac Injection 250mg false false 0.0 0.0 161 90.0
SKU017 Antibiotic_Drug_SKU017 Antibiotic Injection 1g true false 0.0 0.0 179 477.0
SKU018 Analgesic_Drug_SKU018 Analgesic Capsule 1g true false 0.0 0.0 89 201.0
SKU019 Vaccine_Drug_SKU019 Vaccine Capsule 250mg false true 2.0 8.0 175 19.0
SKU020 Analgesic_Drug_SKU020 Analgesic Injection 250mg true false 0.0 0.0 111 459.0
SKU021 Antibiotic_Drug_SKU021 Antibiotic Tablet 1g true true 2.0 8.0 82 347.0
SKU022 Antibiotic_Drug_SKU022 Antibiotic Capsule 500mg true false 0.0 0.0 188 396.0
SKU023 Vaccine_Drug_SKU023 Vaccine Injection 500mg false true 2.0 8.0 162 470.0
SKU024 Antibiotic_Drug_SKU024 Antibiotic Injection 250mg true true 2.0 8.0 200 122.0
SKU026 Analgesic_Drug_SKU026 Analgesic Tablet 1g false false 0.0 0.0 112 411.0
SKU028 Antibiotic_Drug_SKU028 Antibiotic Capsule 250mg true false 0.0 0.0 153 382.0
SKU029 Antibiotic_Drug_SKU029 Antibiotic Tablet 1g false false 0.0 0.0 85 226.0
Time taken: 0.047 seconds, Fetched: 26 row(s)
hive>

```

```

hive> CREATE EXTERNAL TABLE inventory_snapshots_raw (
>     snapshot_date STRING,
>     distributor_id STRING,
>     product_sku STRING,
>     batch_id STRING,
>     opening_stock STRING,
>     receipts STRING,
>     dispatches STRING,
>     adjustments STRING,
>     returns STRING,
>     closing_stock STRING,
>     reorder_point STRING
> )
> ROW FORMAT DELIMITED
> FIELDS TERMINATED BY ','
> STORED AS TEXTFILE
> LOCATION '/user/hadoop/inventory_snapshots'
> TBLPROPERTIES ("skip.header.line.count"="1");

```

OK

Time taken: 0.729 seconds

Time taken: 0.041 seconds

```

hive>
> CREATE TABLE inventory_snapshots_clean AS
> SELECT
>     CAST(snapshot_date AS TIMESTAMP) AS snapshot_date,
>     distributor_id,
>     product_sku,
>     batch_id,
>
>     CAST(opening_stock AS INT) AS opening_stock,
>     CAST(receipts AS INT) AS receipts,
>     CAST(dispatches AS INT) AS dispatches,
>     CAST(adjustments AS INT) AS adjustments,
>     CAST(returns AS INT) AS returns,
>     CAST(closing_stock AS INT) AS closing_stock,
>
>     CAST(NULLIF(TRIM(reorder_point), '') AS INT) AS reorder_point
> FROM inventory_snapshots_raw;

```

Query ID = hadoop_20260410122541_c72c3a7f-dceb-4b51-99cb-f3e3afdc2c28

Total jobs = 1

Launching Job 1 out of 1

Status: Running (Executing on YARN cluster with App id application_1775799336076_0035)

Map 1: --

Moving data to directory hdfs://ip-172-31-27-188.ap-south-1.compute.internal:8020/user/hive/warehouse/inventory_snapshots_clean

OK

Time taken: 3.262 seconds

```

hive>
> CREATE TABLE inventory_snapshots_valid AS
> SELECT *
> FROM inventory_snapshots_clean
> WHERE closing_stock =
>     (opening_stock + receipts - dispatches + returns + adjustments);

```

Query ID = hadoop_20260410122607_17137846-c1f1-4818-9682-d5c0578b04e5

Total jobs = 1

Launching Job 1 out of 1

Status: Running (Executing on YARN cluster with App id application_1775799336076_0035)

Map 1: --

Moving data to directory hdfs://ip-172-31-27-188.ap-south-1.compute.internal:8020/user/hive/warehouse/inventory_snapshots_valid

OK

Time taken: 0.501 seconds

hive>

```

hive> DROP TABLE inventory_snapshots_invalid;

```

OK

Time taken: 0.012 seconds

hive>

```

> CREATE TABLE inventory_snapshots_invalid AS
> SELECT *,
>     'Stock mismatch' AS invalid_reason
> FROM inventory_snapshots_clean
> WHERE closing_stock !=
>     (opening_stock + receipts - dispatches + returns + adjustments);

```

Query ID = hadoop_20260410122631_4800e107-bc82-4611-9b04-1b535de9282c

Total jobs = 1

Launching Job 1 out of 1

Status: Running (Executing on YARN cluster with App id application_1775799336076_0035)

Update

Time taken: 0.211 seconds

```

spark-sql (default)> CREATE TABLE pharma_streaming_db.sales_streaming (
    > transaction_id STRING,
    > pharmacy_id STRING,
    > product_sku STRING,
    > batch_id STRING,
    > quantity_sold INT,
    > unit_price DOUBLE,
    > discount_pct DOUBLE,
    > revenue DOUBLE,
    > fulfillment_status STRING,
    > event_time TIMESTAMP,
    > partition_date DATE
    > )
    > USING PARQUET
    > PARTITIONED BY (partition_date)
    > LOCATION 's3://pharma-data-lake-kritika/streaming_kinesis/sales';

```

26/04/10 13:03:54 WARN SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.security.authorization.manager is set to instance of HiveAuthorizerFactory.

Time taken: 1.081 seconds

```
spark-sql (default)>
```

```

Time taken: 1.309 seconds
hive> aws s3 ls s3://pharma-data-lake-kritika/hive_exports/pharma_db_kritika/
> [hadoop@ip-172-31-27-188 ~]$
[hadoop@ip-172-31-27-188 ~]$ aws s3 ls s3://pharma-data-lake-kritika/hive_exports/pharma_db_kritika/
PRE batch_master_clean/
PRE batch_master_invalid/
PRE batch_master_raw/
PRE batch_master_valid/
PRE inventory_snapshots_clean/
PRE inventory_snapshots_invalid/
PRE inventory_snapshots_raw/
PRE inventory_snapshots_valid/
PRE product_master_clean/
PRE product_master_raw/
PRE product_master_valid/
PRE qc_results_clean/
PRE qc_results_invalid/
PRE qc_results_raw/
PRE qc_results_valid/
[hadoop@ip-172-31-27-188 ~]$

```

aws

Search

[Alt+S] Ask Amazon Q

Asia Pacific (Mumbai)

Account ID: 7761-3744-7724

user10

Amazon Athena > Query editor

Use your IAM role to analyze and build with your existing AWS resources in a single data and AI development environment.

EditorRecent queriesSaved queriesQuery settingsWorkgroupprimary

Data

Data source

AwsDataCatalog

Catalog

None

Database

pharma_db_kritika

Tables and views

Create

Filter tables and views

Tables (7)

< 1 >

Query 6 : XQuery 7 : XQuery 8 : XQuery 9 : XQuery 10 : X

4, payload['sensor_id'] sensor_id

5, payload['batch_id'] batch_id

6, payload['location'] location

7, payload['unit'] unit

8, CAST(payload['temperature_c'] AS DOUBLE) temperature_c

9, CAST(payload['window_avg_c'] AS DOUBLE) window_avg_c

10, CAST(payload['humidity_pct'] AS DOUBLE) humidity_pct

11, CAST(payload['alert_flag'] AS BOOLEAN) alert_flag

12, CAST(payload['anomaly_flag'] AS BOOLEAN) anomaly_flag

13, payload['event_time'] event_time

14, payload['partition_date'] partition_date

15, batch_ts

16 FROM

17 alvis_pharma.temperature_events_stream

18

SQLLn 18, Col 1

CloudShellFeedbackConsole Mobile App© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

Amazon Athena > Query editor

Query in Amazon SageMaker Unified Studio

Use your IAM role to analyze and build with your existing AWS resources in a single data and AI development environment.

Set up

EditorRecent queriesSaved queriesQuery settingsWorkgroupprimary

Data

Data source

AwsDataCatalog

Catalog

None

Database

pharma_db_kritika

Tables and views

Create

Filter tables and views

Query 6 : XQuery 7 : XQuery 8 : XQuery 9 : XQuery 10 : XQuery 11 : X

1 CREATE EXTERNAL TABLE `inventory\_snapshot` (

2 `snapshot\_date` string,

3 `distributor\_id` string,

4 `product\_sku` string,

5 `batch\_id` string,

6 `opening\_stock` int,

7 `receipts` int,

8 `dispatches` int,

9 `adjustments` int,

10 `returns` int,

11 `closing\_stock` int,

12 `reorder\_point` int)

13 ROW FORMAT SERDE

14

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

user10

Amazon Athena > Query editor

EditorRecent queriesSaved queriesQuery settingsWorkgroupprimary

Data

Data source

AwsDataCatalog

Catalog

None

Database

pharma_db_kritika

Tables and views

Create

Filter tables and views

Tables (7)

< 1 >

Query 8 : XQuery 9 : XQuery 10 : XQuery 11 : XQuery 12 : X

1 CREATE EXTERNAL TABLE `product\_master` (

2 `product\_sku` string,

3 `product\_name` string,

4 `generic\_name` string,

5 `category` string,

6 `dosage\_form` string,

7 `strength` string,

8 `schedule` string,

9 `controlled` boolean,

10 `cold\_chain\_required` boolean,

11 `min\_temp\_c` double,

12 `max\_temp\_c` double,

13 `reorder\_point` int,

14 `unit\_cost` double,

15 `unit\_price` double,

SQLLn 1, Col 1

RunExplain L2CancelClearCreate

Reuse query results
up to 60 minutes ago

retailp: X

Kinesi: X

Instan: X

Conn: X

pharm: X

Query X

Query X

Quick X

new-p: X

new-p: X

pharm: X

Maver: X

https://ap-south-1.quicksight.aws.amazon.com/sn/account/mumbaiacct/start/data?tab=dataset

Update ...

Chat

ExploreAmazon QuickDatasets

mumbaiacct

Search

Q

Home

Favorites

QUICK SIGHT

Analyses

Dashboards

Scenarios

Stories

Topics

Datasets

My folders

Shared folders

Prep

Comb

analy

Create

New Amazon Athena data source

X

Data source name

report 1-2

Athena workgroup

[primary]

Validate connection

SSL is enabled

Create data source

Datasets

Data sources

Create dataset

Search datasets by name

Q

Name	Owner	Last Modified	Action
 vw_top_products	 Me	3 days ago	
 vw_payment_revenue	 Me	3 days ago	

Download

